DOCUMENTATION TECHNIQUE

# HealthAI MSPR

Plateforme de suivi santé intelligent

**Projet :** HealthAI MSPR — Monitoring Santé & Recommandations Personnalisées

**Organisation :** EPSI

**Version :** 1.0.0

**Date :** Mars 2026

**Stack :** Vue 3 · FastAPI · PostgreSQL · Docker · Apache Airflow

# Table des matières


| | | |
|---|---|---|
| 1 | Présentation du projet | 3 |
| 2 | Architecture technique | 4 |
| 3 | Stack technologique | 5 |
| 4 | Structure du projet | 6 |
| 5 | Base de données | 7 |
| 6 | API REST — Endpoints | 8 |
| 7 | Authentification JWT | 10 |
| 8 | Frontend — Interface utilisateur | 11 |
| 9 | Pipeline de données — Apache Airflow | 12 |
| 10 | Déploiement Docker | 13 |
| 11 | Sécurité | 14 |
| 12 | Conclusion & Perspectives | 15 |

# 1 Présentation du projet

HealthAI MSPR est une application web full-stack de suivi et d'analyse de données de santé, conçue pour EPSI. Elle permet la collecte, la visualisation et l'analyse des données physiologiques des utilisateurs (activité physique, alimentation, métriques santé, objectifs).

## Objectifs

- ✓ Centraliser les données de santé des utilisateurs
- ✓ Offrir un tableau de bord d'analyse pour les administrateurs
- ✓ Permettre aux utilisateurs de consulter leur profil santé
- ✓ Gérer les flux de données et leur validation
- ✓ Détecter et nettoyer les données incohérentes
- ✓ Automatiser les pipelines de données avec Airflow
- ✓ Sécuriser l'accès via authentification JWT
- ✓ Déploiement 100% conteneurisé avec Docker

## Indicateurs clés

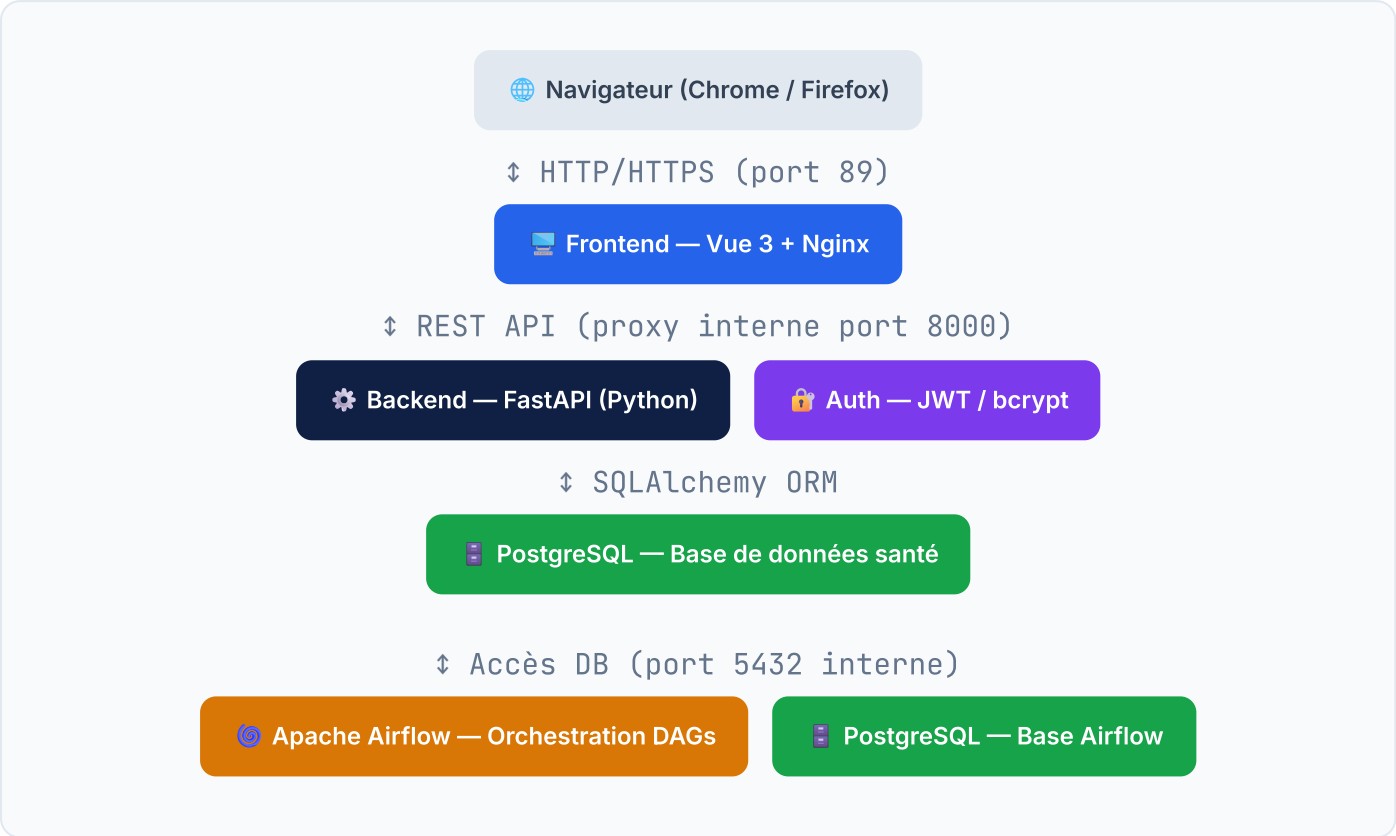
| TABLES DB | ENDPOINTS API | PAGES FRONTEND | SERVICES DOCKER |
|---|---|---|---|
| 7 | 35+ | 8 | 8 |

## Rôles utilisateurs

| Rôle | Accès | Fonctionnalités |
|---|---|---|
| ADMIN | Toutes les pages | Dashboard global, Gestion des flux, Nettoyage des données, Test API, Gestion complète des utilisateurs |
| USER | Page profil uniquement | Consultation du profil personnel, métriques santé, activités, alimentation, objectifs |

# 2 Architecture technique

L'application repose sur une architecture **3-tiers conteneurisée** : présentation (Vue.js + Nginx), logique métier (FastAPI), persistance (PostgreSQL). Un pipeline Airflow orchestre les tâches de données en arrière-plan.




## Flux de communication

| Source | Destination | Protocole | Port |
|---|---|---|---|
| Navigateur | Nginx (Frontend) | HTTP | 89 |
| Nginx (proxy) | FastAPI (Backend) | HTTP interne | 8000 |
| FastAPI | PostgreSQL Health | TCP (SQLAlchemy) | 5432 interne |
| Navigateur (admin) | Airflow Webserver | HTTP | 8080 |
| Airflow | PostgreSQL Health | TCP (psycopg2) | 5432 interne |
| Client externe | PostgreSQL Health | TCP | 5435 |

## Réseau Docker

Tous les conteneurs communiquent via le réseau bridge interne `health_net`. Seuls les ports nécessaires sont exposés vers l'hôte.

## 3 Stack technologique

### Frontend

- **Vue 3** — Framework réactif (Options API)
- **TypeScript** — Typage statique
- **Vite** — Build tool ultra-rapide
- **Vue Router 4** — Navigation SPA
- **Nginx** — Serveur web + proxy

### Backend

- **FastAPI** — Framework Python moderne
- **SQLAlchemy** — ORM Python
- **Pydantic v2** — Validation des données
- **python-jose** — Génération JWT
- **passlib/bcrypt** — Hash des mots de passe

### Données

- **PostgreSQL 17** — Base de données principale
- **Apache Airflow 2.10** — Orchestration DAGs
- **psycopg2** — Driver Python/PostgreSQL
- **CSV seeds** — Données initiales

### Infrastructure

- **Docker** — Conteneurisation
- **Docker Compose** — Orchestration multi-conteneurs
- **Python 3.11-slim** — Image base backend
- **Node 20 + Alpine** — Image build frontend

### Sécurité

- **JWT (HS256)** — Tokens d'accès (8h)
- **bcrypt** — Hash des mots de passe
- **OAuth2 Password Flow** — Standard d'auth
- **RBAC** — Contrôle d'accès par rôle
- **CORS** — Politique d'origine croisée

# 4 Structure du projet

## Backend (Python / FastAPI)

```
backend/ ├── Dockerfile ├── requirements.txt
└── app/ ├── main.py # Point d'entrée API ├──
database.py # Connexion SQLAlchemy ├──
security.py # JWT + bcrypt ├──
create_admin.py # Init compte admin ├──
models/ | ├── utilisateur.py | ├── activite.py
| ├── aliment.py | ├── consommation.py | ├──
exercice.py | ├── metrique_sante.py | └──
objectif.py ├── routers/ | ├── utilisateurs.py
| ├── activites.py | ├── aliments.py | ├──
consommations.py | ├── exercices.py | ├──
metriques_sante.py | └── objectifs.py └──
schemas/ # Pydantic schemas
```

## Frontend (Vue 3 / TypeScript)

```
Frontend/ ├── Dockerfile.nginx ├──
vite.config.ts └── src/ ├── main.ts # Entrée
Vue App ├── App.vue # Layout root ├──
router.ts # Navigation + guards ├──
styles.css # Design System global ├──
components/ | ├── Navbar.vue | └──
AppFooter.vue ├── pages/ | ├── Connexion.vue |
├── PageAccueil.vue # Profil (tabs) | ├──
Dashboard.vue | ├── GestionFlux.vue | ├──
FluxDetail.vue | ├── Nettoyage.vue | ├──
TestBackend.vue | └── PageView.vue └──
services/ ├── auth.ts # JWT + localStorage └──
api.ts # Appels API
```

## Données & Orchestration

```
airflow/dags/ ├── export_db_to_csv.py # DAG : Export PostgreSQL → CSV └──
fetch_openfoodfacts_france.py # DAG : Collecte données OpenFoodFacts data/ ├── seed_db.py #
Chargement données initiales └── clean_csv_data.py # Nettoyage fichiers CSV
```

# 5 Base de données

La base de données `healthdb` sur PostgreSQL 17 contient **7 tables** modélisant le domaine santé utilisateur.

## 👤 utilisateurs — Table centrale des profils

| | | |
|---|---|---|
| **id_utilisateur**<br>INTEGER<br>🔑 PK | **username**<br>VARCHAR(50) UNIQUE | **password_hash**<br>VARCHAR(255) |
| **age**<br>INTEGER | **sexe**<br>VARCHAR(1) | **taille_cm**<br>INTEGER |
| **poids_kg**<br>INTEGER | **niveau_activite**<br>INTEGER | **type_abonnement**<br>INTEGER |
| **date_inscription**<br>DATE | **is_admin**<br>BOOLEAN | |

## ❤️ metriques_sante

`id_metrique` PK · `date_mesure` · `poids_kg` · `frequence_cardiaque` · `duree_sommeil_h` · `calories_brulees` · `pas` · `id_utilisateur` FK

## 🏃 activites

`id_activite` PK · `date_activite` · `duree_minutes` · `calories_depensees` · `intensite` · `id_exercice` FK · `id_utilisateur` FK

## 🎯 objectifs

`id_objectif` PK · `type_objectif` · `description` · `date_debut` · `date_fin` · `statut` · `id_utilisateur` FK

## 🍽️ consommations

`id_consommation` PK · `date_consommation` · `quantite_g` · `calories_calculees` · `id_aliment` FK · `id_utilisateur` FK

## 🍎 aliments

`id_aliment` PK · `nom_aliment` · `calories` · `proteines_g` · `glucides_g` · `lipides_g` · `categorie`

## 💪 exercices

`id_exercice` PK · `nom_exercice` · `type_exercice` · `niveau_difficulte` · `equipement` · `muscle_principal`

# 6 API REST — Endpoints

L'API est documentée automatiquement via **Swagger UI** à l'adresse `http://localhost:8089/docs` (accès direct backend) et proxifiée par Nginx sur le port 89. Toutes les routes (sauf `/login`) requièrent un token JWT Bearer.

## Authentification

| Méthode | Route | Accès | Description |
|---|---|---|---|
| POST | `/login` | Public | Connexion — retourne un token JWT (OAuth2 Password Flow) |
| POST | `/utilisateurs/register` | Public | Inscription d'un nouvel utilisateur |

## Utilisateurs — `/utilisateurs`

| Méthode | Route | Accès | Description |
|---|---|---|---|
| GET | `/utilisateurs/` | Auth | Liste tous les utilisateurs |
| GET | `/utilisateurs/{id}` | Auth | Détail d'un utilisateur par ID |
| POST | `/utilisateurs/` | Admin | Créer un utilisateur |
| PUT | `/utilisateurs/{id}` | Admin | Modifier un utilisateur |
| DELETE | `/utilisateurs/{id}` | Admin | Supprimer un utilisateur |

## Activités — `/activites` & Métriques — `/metriques-sante`

| Méthode | Route | Accès | Description |
|---|---|---|---|
| GET | `/activites/` | Auth | Liste toutes les activités physiques |
| GET | `/activites/{id}` | Auth | Détail d'une activité |
| POST | `/activites/` | Auth | Enregistrer une activité |
| PUT | `/activites/{id}` | Auth | Modifier une activité |
| DELETE | `/activites/{id}` | Admin | Supprimer une activité |
| GET | `/metriques-sante/` | Auth | Liste toutes les métriques santé |
| POST | `/metriques-sante/` | Auth | Enregistrer une mesure santé |

## Alimentation — `/aliments` · `/consommations` · `/objectifs` · `/exercices`

| Méthode | Route | Accès | Description |
|---|---|---|---|
| GET | `/aliments/` | Auth | Liste du catalogue d'aliments (OpenFoodFacts) |
| GET | `/consommations/` | Auth | Historique des consommations alimentaires |
| POST | `/consommations/` | Auth | Enregistrer une consommation |
| GET | `/objectifs/` | Auth | Liste les objectifs santé |
| POST | `/objectifs/` | Auth | Créer un objectif |
| GET | `/exercices/` | Auth | Catalogue des exercices physiques |

Chaque ressource implémente complètement les opérations CRUD (GET, GET/{id}, POST, PUT/{id}, DELETE/{id}). Seules les suppressions sont réservées aux administrateurs.

# 7 Authentification JWT

## Flux d'authentification

1. Saisie identifiants → 2. POST /login form-urlencoded → 3. Vérification bcrypt hash → 4. Génération JWT HS256 → 5. Token stocké localStorage → 6. Bearer token dans chaque requête

## Structure du token JWT

### Header

```
{ "alg": "HS256", "typ": "JWT" }
```

### Payload

```
{ "sub": "1001", // ID utilisateur "is_admin": false, // Rôle "exp": 1741945200 // +8 heures }
```

### Signature

```
HMACSHA256( base64(header) + base64(payload), SECRET_KEY // .env )
```

## Paramètres de sécurité

| Paramètre | Valeur | Rôle |
|---|---|---|
| `ALGORITHM` | HS256 | Algorithme de signature symétrique |
| `ACCESS_TOKEN_EXPIRE_MINUTES` | 480 (8h) | Durée de validité du token |
| `SECRET_KEY` | Variable d'environnement | Clé secrète de signature (non exposée) |
| `pwd_context` | bcrypt (rounds auto) | Hash adaptatif des mots de passe |

## Gestion côté frontend

- Le token est décodé côté client pour extraire `sub` (user ID) et `is_admin`
- `isAuthenticated()` vérifie à la fois la présence du token ET son expiration (champ `exp`)
- Toute réponse `401` provoque un logout automatique et une redirection vers `/connexion`
- Le token est stocké dans `localStorage` sous la clé `app_token`
- Les guards de navigation Vue Router bloquent l'accès aux routes protégées si non authentifié

# 8 Frontend — Interface utilisateur

## Déploiement production (Docker)

Le frontend est servi en **production uniquement** par le conteneur `frontend_nginx`. Aucun serveur de développement Vite ne tourne dans Docker.

- **Build :** `Dockerfile.nginx` — Stage 1 : Vite build (Node 20). Stage 2 : Nginx Alpine sert les fichiers statiques de `dist/`
- **Port :** 89 (hôte) → 80 (conteneur). Accès : `http://localhost:89`
- **Proxy API :** Nginx proxifie `/login`, `/admin`, `/utilisateurs`, etc. vers le backend (même origine = pas de CORS)
- **Variables build :** `VITE_API_URL` (vide = proxy), `VITE_AIRFLOW_UI_URL` (lien vers Airflow)

## Pages et composants

| Page | Route | Accès | Description |
|---|---|---|---|
| **Connexion** | `/connexion` | Public | Formulaire login avec gradient, spinner, toggle mot de passe |
| **Page Accueil** | `/page-accueil` | Auth | Admin : 4 tuiles de nav. Utilisateur : profil avec 5 onglets |
| **Dashboard** | `/dashboard` | Admin | Top 3 exercices/aliments, moyennes utilisateurs |
| **Gestion Flux** | `/gestion-des-flux` | Admin | CSV intermédiaires (import/export) à valider, métadonnées BDD, lien vers DAGs Airflow |
| **Flux Détail** | `/flux/:id` | Admin | Détail d'un flux avec actions |
| **Nettoyage** | `/nettoyage` | Admin | Détection automatique de données incohérentes |
| **Test Backend** | `/test-backend` | Admin | Exploration complète des endpoints et données brutes |
| **Utilisateurs** | `/utilisateurs` | Admin | Gestion des utilisateurs (recherche, détail) |
| **Page View** | `/page-view/:id?` | Public | Maquette interactive des pages |

## Profil utilisateur — Navigation par onglets

| 👤 | ❤️ | 🏃 | 🍽️ |
|---|---|---|---|
| ONGLET 1 | ONGLET 2 | ONGLET 3 | ONGLET 4 |
| **Profil** | **Santé** | **Activités** | **Alimentation** |

## Design System

- **Police :** Inter (Google Fonts) — 400, 500, 600, 700, 800
- **Couleurs primaires :** Navy `#0f2044` / Blue `#2563eb` / Gray Scale

- **Variables CSS :** Système complet de tokens (radius, shadow, transition, couleurs)
- **Responsive :** Breakpoints à 500px, 768px, 900px
- **Navbar :** Gradient navy → blue, sticky, liens actifs surlignés, badge rôle
- **Footer :** Global, marque + liens + version, fond navy

## Nettoyage automatique des données (page Admin)

La page Nettoyage analyse en temps réel les données via l'API et signale :

- Objectifs avec statut refusé ou rejeté
- Objectifs sans description
- Consommations avec calories ou quantité invalides (≤ 0)
- Métriques avec fréquence cardiaque incohérente (< 30 ou > 220 bpm)
- Métriques avec poids incohérent (< 20 ou > 350 kg)

# 9 Pipeline de données — Apache Airflow

Apache Airflow 2.10 orchestre les pipelines de données asynchrones. Interface web sur le port **8080** (identifiants : airflow / airflow). Le backend accède à l'API Airflow via `host.docker.internal:8080` .

## DAG 1 — Import Open Food Facts

| Attribut | Valeur |
|---|---|
| Fichier | `airflow/dags/fetch_openfoodfacts_france.py` |
| Source | API OpenFoodFacts (produits alimentaires France) |
| Sortie | `data/intermediate/import/import_YYYYMMDD_HHMMSS.csv` (validation admin requise) |
| Objectif | Collecter des aliments depuis l'API Open Food Facts vers un CSV intermédiaire |

## DAG 2 — Export base de données vers CSV

| Attribut | Valeur |
|---|---|
| Fichier | `airflow/dags/export_db_to_csv.py` |
| Connexion DB | `AIRFLOW_CONN_HEALTHAIM_POSTGRES` |
| Sortie | `data/intermediate/export/export_YYYYMMDD_HHMMSS.csv` (validation admin requise) |
| Objectif | Exporter les tables de healthdb vers un CSV intermédiaire |

## DAG 3 — Incorporation ML

| Attribut | Valeur |
|---|---|
| Fichier | `airflow/dags/incorporation_ml.py` |
| Fréquence | Toutes les heures |
| Entrée | CSV validés dans `data/intermediate/import/` et `data/intermediate/export/` |
| Sortie | Import → BDD healthdb ; Export → `data/ml/` |
| Objectif | Charger les CSV validés par l'admin en base et copier les exports vers le dossier ML |

## Flux de validation

Les DAGs produisent des CSV horodatés. L'admin visualise, modifie et valide via la page **Gestion des flux** (onglets Import/Export). Seuls les CSV validés sont incorporés par l'DAG `incorporation_ml` .

## Infrastructure Airflow

**airflow-init**

**airflow-webserver**

**airflow-scheduler**

Initialisation de la base Airflow et création du compte admin (exécution unique)

Interface web de monitoring des DAGs (port 8080)

Planificateur d'exécution des DAGs (LocalExecutor)

# 10 Déploiement Docker

## Services Docker Compose

| Conteneur | Image | Port exposé | Rôle |
|---|---|---|---|
| `frontend_nginx` | healthaimspr-frontend (custom) | 89:80 | Serveur Vue.js + proxy API |
| `backend_api` | healthaimspr-backend (custom) | 8089:8000 | API FastAPI + Uvicorn |
| `postgres_health` | postgres:17 | 5435:5432 | Base de données applicative |
| `airflow_webserver` | apache/airflow:2.10.4-python3.11 | 8080:8080 | UI monitoring Airflow |
| `airflow_scheduler` | apache/airflow:2.10.4-python3.11 | — | Planificateur de DAGs |
| `postgres_airflow` | postgres:17 | — | Métadonnées Airflow |
| `export_init` | alpine | — | Init permissions export folder |

## Variables d'environnement (.env)

```
# Base de données principale
DATABASE_URL=postgresql://healthuser:healthpass@postgres:5432/healthdb # Sécurité API
SECRET_KEY=votre-clé-secrète-forte # Ports (optionnels, valeurs par défaut)
FRONTEND_PORT=127.0.0.1:89 BACKEND_PORT=127.0.0.1:8089 POSTGRES_PORT=127.0.0.1:5435
AIRFLOW_PORT=8080 # Frontend (URL de l'API depuis le navigateur) VITE_API_URL=http://localhost:89
```

## Commandes de déploiement

| Commande | Action |
|---|---|
| `docker compose up -d postgres` | Démarre la base de données PostgreSQL utilisée par l'application |
| `docker compose --profile seed run --rm seed` | Charge les données initiales (utilisateurs, aliments, exercices, etc.) dans la base |
| `docker compose --profile admin run --rm admin` | Création d'un compte admin (identifiants : admin / admin) pour accéder à l'interface d'administration et valider les flux Airflow |
| `docker compose --profile airflow run --rm airflow-init` | Initialise la base de données Airflow et crée le compte administrateur (exécution unique) |
| `docker compose --profile airflow up -d` | Démarre l'ensemble des services : frontend, backend API, Airflow et base de données |
| `docker compose up -d --build` | Démarrer tous les services en arrière-plan |
| `docker compose logs -f backend_api` | Suivre les logs du backend en temps réel |

| Commande | Action |
| --- | --- |
| `docker compose down` | Arrêter tous les services |

# 11 Sécurité

## Authentification & Autorisation

- **JWT HS256** — Tokens signés, expiration 8h
- **bcrypt** — Hash adaptatif des mots de passe, jamais stockés en clair
- **OAuth2 Password Flow** — Standard RFC 6749
- **RBAC** — 2 rôles : `user` et `admin`
- Vérification du token à chaque endpoint protégé
- Déconnexion automatique si token expiré (frontend)

## Protection des endpoints

- Routes publiques : `/login`, `/utilisateurs/register`
- Routes auth : lecture des données (GET)
- Routes admin-only : suppressions (DELETE) et modifications globales
- Validation stricte avec **Pydantic v2** sur toutes les entrées
- Path parameters validés ( `gt=0` pour les IDs)

## Infrastructure

- Ports DB exposés uniquement sur `127.0.0.1`
- `SECRET_KEY` via variable d'environnement (jamais en dur)
- Réseau Docker interne isolé ( `health_net` )
- CORS configuré (à restreindre en production)

## Frontend

- Guards de navigation empêchent l'accès non autorisé
- Vérification expiration JWT côté client (champ `exp` )
- Gestion des 401 : logout + redirection automatique
- Pas de données sensibles exposées dans le DOM

## Limites et améliorations recommandées

| Risque actuel | Recommandation |
|---|---|
| Token stocké en `localStorage` (vulnérable XSS) | Migrer vers `httpOnly cookies` en production |
| CORS `allow_origins=["*"]` | Restreindre à l'origine frontend réelle |
| Pas de refresh token | Implémenter un mécanisme de renouvellement silencieux |
| Pas de rate limiting sur `/login` | Ajouter une limite de tentatives (brute force) |

# 12 Conclusion & Perspectives

## Bilan du projet

HealthAI MSPR est une plateforme de suivi santé complète, fonctionnelle et entièrement conteneurisée. Elle couvre l'ensemble des besoins exprimés : gestion des utilisateurs, suivi des données physiologiques, analyse statistique, détection d'anomalies, et orchestration de pipelines de données.

**RÉALISÉ**

API REST complète · Auth JWT · Dashboard admin · Profil utilisateur par onglets · Détection anomalies · Pipeline Airflow · Design system moderne · Footer global · Déploiement Docker

**PERSPECTIVES**

Recommandations IA personnalisées · Graphiques de tendances · Notifications temps réel · Application mobile · Export PDF des rapports · Refresh tokens · Tests automatisés complets

**QUALITÉ TECHNIQUE**

Architecture découplée · TypeScript strict · ORM typé · Validation Pydantic v2 · Tests backend disponibles · Documentation auto Swagger

## HealthAI MSPR

Plateforme de suivi santé intelligent — EPSI · 2026

Vue 3 + TypeScript · FastAPI + Python · PostgreSQL 17 · Docker Compose · Apache Airflow